

Configuração e do ambiente virtual (venv)

A seguir vou detalhar cada passo necessário para criar um ambiente virtual. O último item é uma lista corrida dos comandos, sem explicação.

1. Criar a venv

Também chamado de virtual environment, ou ambiente virtual.

Abra um terminal na raiz do projeto, ou seja, na pasta do projeto. Por exemplo, se o nome do meu projeto for “material_aula”, o terminal deve mostrar algo como:

```
C:\Users\material_aula>
```

ATENÇÃO!

Esse caminho é um exemplo, você deve estar no caminho do SEU PROJETO! Para entender o que é caminho no windows, verifique esses links: [aqui](#) e [aqui](#).

Feito isso, crie a pasta .venv contendo o ambiente isolado do projeto com o seguinte comando:

```
python -m venv .venv
```

Para entender o comando:

- `python` executa o interpretador python, teste `python --help` para descobrir outros comandos possíveis
- `-m` executa um módulo de python como um programa, módulos são pequenos scripts, tipo bibliotecas, criadas para que uma sequência de comandos python sejam executados automaticamente
- `venv` é um módulo que cria ambientes virtuais
- `.venv` é o nome do ambiente que será criado, ou seja, você pode dar qualquer nome que queira. O `.`` não é obrigatório, mas ele indica que a pasta será oculta.

2. Ativar a venv

No mesmo terminal, ative o ambiente com o comando abaixo. Caso você dê outro nome para o ambiente, substitua `.venv` pelo nome da pasta.

```
.\.venv\Scripts\Activate.ps1
```

Após a ativação, o terminal deverá exibir algo semelhante a:

```
(.venv) PS C:\caminho\projeto>
```

3. Selecionar a venv no VS Code

Pressione `Ctrl + Shift + P` e procure por `Python: Select Interpreter`. Selecione o interpretador `.\venv\Scripts\python.exe`.

Alternativamente, clique no botão no canto direito superior da tela, onde está escrito `kernel`, ou o nome de algum outro ambiente, e selecione `.\venv\Scripts\python.exe`, ou o `Mais` e procure o interpretar do seu ambiente.

Repare estamos configurando o caminho para o interpretador do python. O caminho `.\venv\Scripts\` é o endereço, o path, onde o python e as bibliotecas estão salvas, e `python.exe` é o arquivo executável que roda o python. É ele que interpreta seu código e executa os comandos dos seus scripts. Se você ficou interessado nisso, pesquise mais sobre a diferença entre linguagens interpretadas e compiladas, ou leia [aqui](#).

4. Instalar dependências

Com a venv ativada, instale as dependências desejadas:

```
pip install nome_do_pacote
```

Exemplos de pacotes são `matplotlib`, `pandas`, etc.

Podemos também juntar vários pacotes em um `pip` só:

```
pip install pandas numpy matplotlib
```

5. Gerar o arquivo requirements.txt -> para poder versionar

Após instalar todas as dependências necessárias (pacotes, libs, instaladas), vamos criar um arquivo que registra o nome deles:

```
pip freeze > requirements.txt
```

Para visualizar o conteúdo:

```
type requirements.txt
```

6. Versionar o requirements.txt -> enviar para o Repositório git

Esse arquivo não estará no `.gitignore`, ou seja, ele não será ignorado. Ele é um pequeno arquivo de texto que indica para um sistema quais dependências (libs) são necessárias no projeto.

Faremos `add`, `commit` e `push` nele assim como em qualquer outro arquivo.

Caso queira fazer diretamente pelo terminal, podemos fazer:

```
git add requirements.txt
git commit -m "Atualiza dependências do projeto"
git push
```

Isso é o mesmo que subir no git via vscode ou via github desktop.

Obs: o ambiente virtual não é adicionado ao repositório porque isso significaria subir em cada repositório uma série de bibliotecas que já estão disponíveis para download em seus respectivos repositórios. Por isso, o cache do python e do ambiente devem estar no arquivo `.gitignore`. Em geral isso é feito automático. Quando não é, o arquivo pode ser manualmente criado com o conteúdo:

```
.venv/
__pycache__/
*.pyc
```

7. Configurar o segundo computador -> Ao clonar o repositório

Após clonar ou atualizar o repositório pela primeira vez, devemos criar e ativar um ambiente virtual na nova máquina. Os comandos usados serão os mesmos de 1. e 2.:

```
python -m venv .venv
```

E

```
.\.venv\Scripts\Activate.ps1
```

Por fim, devemos instalar as dependências com

```
pip install -r requirements.txt
```

Pronto!

Fluxo de trabalho

Obs: `.\.venv\Scripts\Activate.ps1` pode ser substituído por escolher o ambiente pela interface do vscode.

Primeira vez em cada computador

```
python -m venv .venv  
.\.venv\Scripts\Activate.ps1  
pip install -r requirements.txt
```

Após atualização do repositório (`git pull`)

```
.\.venv\Scripts\Activate.ps1  
pip install -r requirements.txt
```

Este comando só atualizará dependências novas ou alteradas.